

ParaView-MCP: An Autonomous Visualization Agent with Direct Tool Use

Shusen Liu

Haichao Miao

Peer-Timo Bremer

CASC, Computing, Lawrence Livermore National Laboratory
 {liu42, miao1, bremer5}@llnl.gov

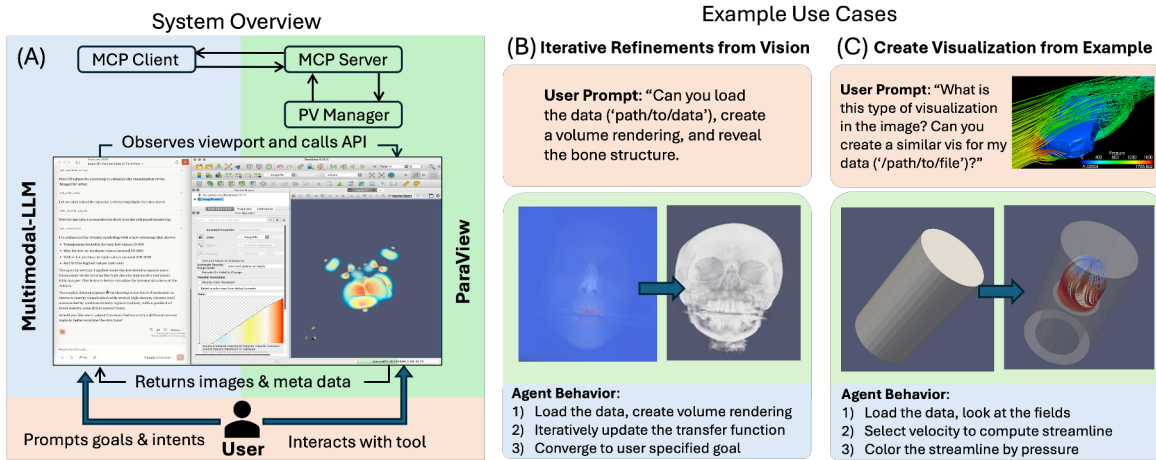


Figure 1: ParaView-MCP brings together the users, the tool, and the agent into a collaborative environment, where users can either express goals via the agent to achieve the visualization objective or directly interact with the ParaView GUI. (A) System Overview: We encapsulated ParaView's API via the ParaView Manager and developed an MCP Server to allow a multimodal large language model (MLLM) to control ParaView directly. By combining the MLLM's and ParaView's capabilities, we show novel usages that were not possible before with these tools alone, as shown in the examples (B, C).

ABSTRACT

While powerful and well-established, tools like ParaView present a steep learning curve that can discourage many potential users. This work introduces ParaView-MCP, an autonomous agent that integrates modern multimodal large language models (MLLMs) with ParaView to not only lower the barrier to entry but also augment ParaView with intelligent decision support. By leveraging the state-of-the-art reasoning, command execution, and vision capabilities of MLLMs, ParaView-MCP enables users to interact with ParaView through natural language and visual inputs. Specifically, our system adopted the Model Context Protocol (MCP), a standardized interface for model-application communication, which facilitates direct interaction between MLLMs and ParaView's Python API, allowing seamless information exchange between the user, the language model, and the visualization tool itself. Furthermore, by implementing a visual feedback mechanism that allows the agent to observe the viewport, we unlock a range of new capabilities, including recreating visualizations from examples, closed-loop visualization parameter updates based on user-defined goals, and even cross-application collaboration involving multiple tools.

Index Terms: Agent, Tool Use, Model Context Protocol

1 INTRODUCTION

Scientific Visualization (SciVis) plays a crucial role in modern data-driven research and engineering workflows. Tools such as ParaView [2] are widely recognized for their powerful capabilities,

scalability, and flexibility across a broad range of application domains. A key barrier to adoption lies in the opportunity cost and steep learning curve [5] associated with mastering these tools. While ParaView itself is designed to leverage a GUI interface to improve the usability [3] the complexity of the vast functions still introduce challenges. The domain users often cannot justify the time and effort required to learn a new tool, especially when their focus lies elsewhere. Task such as transfer function design for volume visualization has been cited [18] as non-intuitive and time-consuming. Anecdotal, several application scientists we work with express sentiments such as: "I plan to learn to use it, but I never got around to it." The threshold for entry will be significantly lowered if someone is available to guide them hands-on, using their own data and use cases.

With the advent of modern multimodal large language models (MLLMs), new opportunities arise for improving tool usability. These models not only possess broad general knowledge but can also issue structured commands and follow complex, goal-oriented instructions. This opens up exciting possibilities for rethinking how users interact with software, not through complex and hard to master traditional user interfaces, but via conversational, intelligent agents that can understand user intent and act on their behalf.

In this work, we present ParaView-MCP, an *autonomous visualization agent* that integrates a state-of-the-art MLLM with the ParaView platform. Our system leverages the Model Context Protocol (MCP) [7], a model-application exchange standard, and ParaView's Python API to establish a direct, bidirectional communication pipeline between the user, the language model, and the visualization tool. This integration allows for seamless command execution, visualization manipulation through natural language, and context-aware interactions. Through this setup, users can perform tasks that previously required extensive training and/or manual ef-

fort. For instance, the agent can recreate visualizations from examples, automatically update plots based on user-defined goals, or even coordinate across multiple tools in a collaborative analysis scenario. These capabilities illustrate how AI-driven agents can significantly extend the usability and expressiveness of existing software systems.

Ultimately, ParaView-MCP aims to lower the barrier to entry for high-quality scientific visualization, transforming how users access, control, and benefit from complex visualization tools. This paper outlines the system’s design, implementation, and capabilities and highlights a set of case studies that demonstrate its potential in real-world scenarios. By facilitating direct tool use via MCP-enabled multimodal agents, our work unites three previously disparate threads that make decades of SciVis algorithms embedded in ParaView more accessible to domain users via conversational interfaces that democratize the visualization of complex datasets.

2 RELATED WORK

Making visualization tools accessible through natural language has been a long-standing goal. Early systems such as *DataTone* handled linguistic ambiguity via interactive widgets [10]; *Eviza* enabled follow-up Q&A and incremental view refinement in conversational form [26]. Subsequent toolkits (NL4DV, FlowSense, etc.) broadened the mapping from natural language to Vega-Lite or dataflow specifications [21, 30]. With the recent advancement, AI-centric visualization techniques [31, 37] have grown dramatically and spread through most subfields of visualization, e.g., visualization education [14], visualization design/recommendation [12, 29, 28, 9, 33, 23], compression [19], and super-resolution [11]. Moreover, the visual-understanding capabilities of modern MLLMs have sparked studies for evaluating how well these models interpret visualization [35, 8, 16, 27, 34]. Although results reveal notable shortcomings, the overall trend points to rapid improvement, providing growing confidence that MLLMs can supply meaningful visual reasoning.

Another significant change comes from research on AI agents [24], where autonomous decisions are carried out based on human preferences. Visualization researchers have previously explored agent-like systems: *VisComplete* auto-completes visualization pipelines using histories [15]; *ChatVis* iteratively refined ParaView Python scripts with an LLM and error-feedback loop [20]; The *MatplotlibAgent* [36] work assesses various models’ ability to generate matplotlib code for general visualization tasks; *AVA* optimized volume-rendering parameters through closed-loop optimization [17]. While compelling, these systems stop short of direct in-situ control of an existing tool’s interface during an interactive exploration and analysis session.

3 PARAVIEW-MCP

3.1 Autonomous Visualization Agent Design

Many tools can be controlled through direct code generation, such as those used in *ChatVis* [20]. While this method is relatively simple to implement and effective for experienced users, it is inherently limiting, as it does not support collaborative interaction among the tool, user, and agent. Vision-based computer use, where the agent interacts with the GUI as a human user would (controlling mouse and keyboard), offers a general solution applicable to a wide range of tools. However, current implementations, such as OpenAI’s Operator [22] or Anthropic’s Computer Use [6], lack the reliability and speed necessary for effective interactive operation. The slow response times and challenges in navigating complex interfaces prevent this approach from being a viable solution for our goal.

Our approach leverages the strengths of direct API integration while focusing the model’s vision capability where it is needed most, i.e., interpreting the visualization in the viewport. By utilizing ParaView’s existing Python API, the agent can directly manipu-

late the tool without requiring any alteration to its software architecture. This hybrid design enables interactive operation where both the user and the agent can collaboratively manipulate the visualization and observe the updates. Moreover, the visual feedback further reduces potential hallucination issues by grounding the agent’s decisions in the visualization results. While natural language interaction is a key component, the ability to iteratively refine actions based on visualization results ensures accuracy and precision.

3.2 System Implementation

The hybrid scheme discussed above is achieved by creating and exposing a structured tool abstraction of ParaView to the language-model via an MCP server.

ParaView Control Via Multi-Client PvServer ParaView offers a comprehensive Python API, `paraview.simple`, which enables programmatic access to nearly all aspects of the tool—from data loading and processing to visualization workflows and rendering. To allow control of ParaView from both the GUI interface as well as the agent while maintaining interactivity, we utilize ParaView’s `pvsrvr` in multi-client mode. In this setup, both the user (through the ParaView GUI) and the autonomous agent (through ParaView-MCP Server connected to the same `pvsrvr`) can issue commands and observe updates in a shared session. This enables seamless, collaborative control over the same visualization state without sacrificing user agency or GUI responsiveness.

ParaView-MCP Server At the heart of our system lies the ParaView-MCP server, which bridges the language model and the visualization tool. This server is built on the *Model Context Protocol (MCP)* [7], a standardized framework designed to provide LLMs with structured, contextual access to external tools. We utilize the `FastMCP`¹ library to help build the MCP server. The MCP server is agnostic to the underlying LLM, so that our tool is compatible with different model services or locally deployed LLMs, while preserving a consistent and interpretable tool interface. This abstraction makes the system auditable and adaptable as models are ever-increasing in their capabilities. More importantly, the MCP server will provide convenience functions to look up all possible tools and be responsible for injecting the docstrings of the tool functions into the model’s context during inference to help the LLM determine which functions to call.

Exposing the low-level ParaView API directly as functions/tools for the MCP server is not a viable solution: 1) a large number of functions may make selecting the correct one more challenging; 2) the protocol is not designed to work with arbitrary function returns that are native to ParaView. As a result, an additional abstraction is needed by the MCP Server to define the functionality of ParaView that can be triggered by the LLMs. We handcraft functions to support key usages, e.g., data loading, isosurface, volume rendering, streamline, source selection, screenshot, etc. Based on this higher-level abstraction (i.e., the ParaView Manager discussed later), the ParaView-MCP server exposes a curated set of ParaView functionalities to the language model. Specifically, the MCP server will take the docstring and comments of functions inject into the model’s context to helps the LLM decide which functions to call. The LLM will handle the mapping from language to the set of appropriate tools, validate returns, and append information to the LLM context for future reference. There is no training required, and the MCP server is compatible with most existing LLMs.

ParaView Manager The *ParaView Manager* encapsulates the ParaView functionalities that the MCP Server exposes to the LLMs. This ensures reliable execution, avoids unsafe code generation, and supports repeatability—important for scientific reproducibility and debugging. One persistent challenge we encounter when designing this encapsulation is tracking the active object and visualization

¹<https://github.com/jlowin/fastmcp>

state. Ideally, we prefer the abstraction layer to be stateless. To achieve that, we expose specific functions to query existing ParaView *sources* by type or name, select active *sources*, and determine whether the operation can be applied to the source. For the future iterations of ParaView-MCP, dynamic tool synthesis may offer additional flexibility, but for now, we prioritize robustness and traceability. Moreover, the ParaView Manager also facilitates the query of the viewport during the interaction, which facilitates vision-grounded reasoning that sets the proposed approach apart from many existing generative visualization approaches that focus on image and code generation rather than visual assessment or closed-loop iteration. Lastly, we open-sourced the implementation of ParaView-MCP and plan to continue its further development².

4 USE CASES

In this section, we illustrate potential use cases for ParaView-MCP. While these examples highlight interesting functionalities, the flexibility of the system enables a wide range of possibilities beyond what can be covered here. We also refer the reader to the supplementary video³ for the full interaction of these use cases.

4.1 Language to Result with Vision-Guided Refinement

ParaView-MCP translates users' goals and intentions into MCP function calls that control the ParaView GUI. The user can directly ask for specific features, like creating an isosurface or volume rendering of their volumetric data. More interestingly, the agent can pursue user-defined objectives iteratively. As illustrated in Figure 2, a user could specify the goal to identify an isovalue that generates a mesh surface that is roughly half of the original one. ParaView-MCP will then iteratively adjust the iso values until this goal is met.

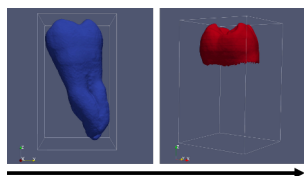


Figure 2: The agent automatically identifies the iso-value that leads to half of the original surface area.

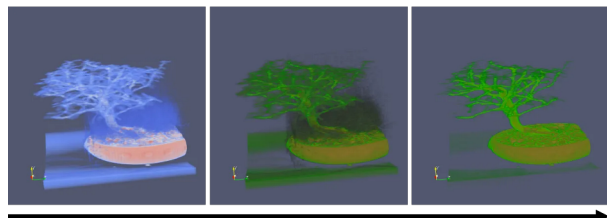


Figure 3: Iterative transfer function design driven by a vision-based feedback loop. The agent starts with the default colormap (*Cold-to-Warm*) to assess the value distribution, and based on user instruction (*green tree, brown base*), designs and refines the colormap by visually examining each rendering output.

Moreover, by leveraging vision-based feedback and function return values, the system enables iterative refinement to achieve user-defined objectives. As shown in Figure 3, the user specifies the target color for both tree and base, the agent follows the instruction, recognizes the structure in the image, and then tries to assign the correct color to the right object, iteratively. Similarly, in Figure 1(b), the agent successfully designs the transfer function that highlights the bone structure. Transfer function design process deals with a large action space, which could benefit from a gradient-based optimization approach such as proposed by Jeong et al. [13]. However, their approach requires not only a fully differentiable volume

renderer but also the CLIP [25] scoring model that has a weaker vision capability compared to the frontier MLLMs (e.g., GPT, Claude).

4.2 Visualization by Example and Co-Exploration

The multimodal capabilities of the MLLM enable users to provide visual references alongside language instructions. For instance, a user could request the agent to replicate a visualization based on a visual example, as illustrated in Figure 1 (c). This ability to combine visual and linguistic inputs enhances the agent's understanding and facilitates the recreation of potentially complex visualization workflows that include multiple tasks (including the application of multiple filters in ParaView). Additionally, the conversational interface of ParaView-MCP facilitates collaborative exploration by allowing novice users to ask questions about their data and potential visualization solutions. As illustrated in Figure 4, the user start by asking the agent to help explain the loaded data, then the agent not only provide a detailed breakdown of the content of the data but also suggest and explain suitable visualization approaches, and work with the user to explore the different type of data/field together.

4.3 Multi-Tool Interaction and Collaboration

Visualization tasks often utilize not one but multiple tools. Here we demonstrate a use case where, with the help of a second MCP server for another tool, an illustrative visualization is generated from a volume dataset. Multiple MCP servers can connect to the same MCP client (e.g., the Claude Desktop App), enabling seamless integration across different tools. As shown in Figure 5, the agent can generate an iso-contour using ParaView-MCP and subsequently send the mesh into Blender via the *Blender MCP server* [4]. This interoperability allows users to leverage the strengths of multiple tools in a unified workflow, facilitating advanced cross-platform analysis and visualization tasks.

5 PRELIMINARY DOMAIN EXPERT FEEDBACK

To obtain rapid, formative insights on the utility and research relevance of ParaView-MCP, we invited two domain experts to review the described prototype. As intended, neither had used ParaView before, but both actively work with CT data and had heard of ParaView. E1 is a research scientist in non-destructive testing who specializes in the acquisition and analysis of CT scans. E2 is a program manager who oversees a portfolio of research projects in additive manufacturing and material characterization. We demonstrated the tool using the transfer function design and isosurface selection examples, and reported their informal feedback below. E1 said *"the fact that it directly controls ParaView is incredible"*. E2 commented that she not only sees the value proposition for novice users, but also experienced ParaView users who can *"use this tool and let it automatically explore a bunch of datasets"*. When demonstrating the transfer function design use case to visualize the bonsai tree like a *real tree*, E1 noted that after the initial prompt, the agent chose an inverted color map, since lower density values, corresponding to leaves, are mapped to brown and higher density values, corresponding to the tree trunk, are mapped to green. With a second prompt *"could you please render the lower density that represents leaves in a different color than the wood and the pot?"* the agent was able to color the leaves in a different color. He found the text explanation from the agent while inspecting the result in ParaView in particularly useful to understand the agent's reasoning and also to learn about ParaView usage. E2 also made an insightful suggestion that the agent should be able to understand all the actions that the user conducted in the ParaView GUI, beyond only observing the viewport, which we plan to support. Both users agreed that this tool lowers the barrier of entry to a complex visualization tool.

²https://github.com/LLNL/paraview_mcp

³See the video demo at: <https://youtu.be/GvcBnAcIXp4>

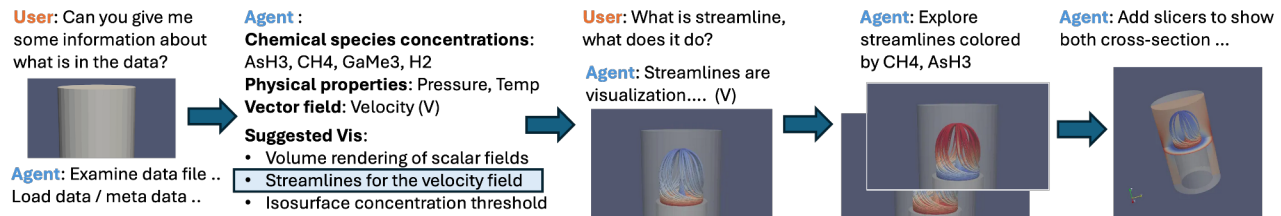


Figure 4: An illustration of the co-exploration use case, where the ParaView-MCP works with a novice user by suggesting a list of possible visualization methods, explaining specific techniques (e.g., streamline), and guiding the overall exploration process.

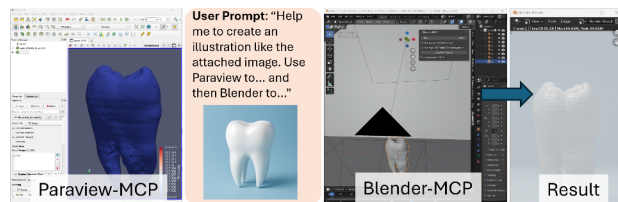


Figure 5: A multi-tool collaboration example use case, where ParaView-MCP and Blender-MCP [4] work together to create an illustration from a volumetric dataset of a tooth. Per user prompt, ParaView-MCP extracts the iso contour and then renders the surface in Blender-MCP based on the example image the user provided.

6 DISCUSSION

Agentic Paradigm for Visualization Since the advent of visualization for medical and scientific use cases, the visualization research community has developed a vast number of custom techniques/tools for ever-changing domain applications. Yet, the actual adoption of these custom visualization methods remains low. ParaView-MCP does not propose a new visualization method, but a visualization agent with a direct tool use paradigm. This paradigm shift could address the “last mile” problem for the adoption of existing visualization methods by significantly lowering the barrier of entry. Specifically, ParaView-MCP can directly control ParaView, observe its viewport, and interact with the user via natural language. To the best of our knowledge, this is the first implementation of the Model Context Protocol (MCP) for a complex SciVis tool, demonstrating its applicability and effectiveness in handling sophisticated visualization workflows. We believe autonomous visualization agents would significantly alter how we interact with tools in the future. We hope to encourage the creation of similar autonomous visualization agents by demonstrating their viability and strengths. At the same time, we also want to discuss their limitations, provide guidance on the design, and cover lessons learned. However, due to the autonomous nature of LLMs, it is important to discuss ethical considerations for future improvements. First, maintaining user agency and oversight is essential, as the agent’s decisions, even when well-intentioned, can still be flawed or misleading, especially in high-stakes scientific contexts. Second, the transparency and reproducibility of model-driven actions must be ensured, since AI-generated code or visualization steps need to be auditable and verifiable to support scientific integrity and trust.

Challenges and Lessons Learned In the current work, ParaView-MCP utilizes Claude’s Sonnet 3.7, which is Anthropic’s most intelligent model. Still, we observed the limitation of this model when controlling ParaView. For example, as noted by E1, it inverted the color map for the bonsai tree even though it should have had general knowledge about the density values of leaves vs. wood. On some occasions, it would oscillate between several wrong color maps without ever reaching the correct solution, even though it observes the changes it made via the viewport. Besides the inherent stochasticity of LLM, erroneous behaviors like that also spark the

question of how intelligent these models are, if they can conduct highly complex tasks yet fail at seemingly common knowledge. Because of cases like this, the human-in-the-loop will remain ultimately crucial in high-consequence workflows. Specifically, the visualization agent paradigm introduced here advocates a collaborative process, where the agent supports the human expert, such as tedious and laborious tasks, with direct tool use, but the user ultimately remains in charge.

For the important lessons learned. First, from our initial experiment, the speed of interaction, seemingly trivial, has a big impact on the user experience. Its importance becomes painfully obvious when we experiment with pure vision-based tool use, i.e., Claude Computer Use API [6]. Besides limited navigation accuracy, a single interaction step (mouse click/movement, then vision feedback from the entire screen) takes somewhere between 10-20 seconds, and opening a menu sometimes will require 3 steps. As a result, we instead focus on designing tool-use strictly through the ParaView API, which is multiple magnitudes faster. Besides interactivity, as discussed in detail in Section 3.2, creating a useful agent goes beyond exposing the API and viewport to the MLLM. Significant considerations have to be made to ensure consistent and predictable behavior of the agent; hence, we iterate different abstraction designs for the ParaView Manager. One general limitation of these MCP agents is that they need to be aware of the state of the software they control. Beyond the VTK pipeline query, we also allow the model to screenshot the viewport to check whether tasks such as transfer function design and the application of filters worked. However, in the future, a more fine-grained two-way communication would be needed that allows the agent to query the history of tasks executed by the user beyond just the current state of ParaView.

Future Directions With our open-source release of ParaView-MCP, we anticipate feedback on its capabilities and limitations to guide future feature implementation. Our immediate focus will be on enhancing batch processing abilities, particularly for opening datasets and creating/deleting sources, facilitating visualization tasks across multiple datasets. One of the prevalent challenges in various models is their ability to robustly select the right function from a large number of exposed MCP tools. Due to the limitation of MCP, all available tools from all MCP servers are exposed to the model as a flat list of functions. To future-proof Paraview-MCP for collaborative tasks, we will integrate it with multi-agent frameworks, e.g., AutoGen [32] and A2A [1].

A reliable API interface forms the necessary foundation. The *multi-client* option isn’t optimal for agent/tool communication, being recently deprecated in ParaView. A more robust solution would be to develop a ParaView plug-in to directly inject commands into its Python console. Looking further ahead, combining MLLM agentic frameworks for high-level planning with dedicated optimizers (e.g., gradient-based TF design [13]) for lower-level tasks presents an intriguing direction, potentially delivering the best of both approaches while improving speed and efficiency. Lastly, we also plan to test Paraview-MCP with on-prem LLMs to test its robustness with smaller models.

ACKNOWLEDGMENTS

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344 (Review and released under LLNL-CONF-2005387). The work was partially supported by the LLNL-LDRD (23-ERD-029, 23-SI-003), and DOE ECRP (SCW1885). We also like to thank Jenny Wang-Nicolino and Massimiliano Ferruci for their thoughtful feedback.

REFERENCES

- [1] a2aproject. A2A: An open protocol enabling communication and interoperability between opaque agentic applications. <https://github.com/a2aproject/A2A>, June 2025. 4
- [2] J. Ahrens, B. Geveci, and C. Law. Paraview: An end-user tool for large data visualization. *Visualization Handbook*, 01 2005. 1
- [3] J. Ahrens, B. Geveci, C. Law, C. Hansen, C. Johnson, et al. 36-paraview: An end-user tool for large-data visualization. *The visualization handbook*, 717:50038–1, 2005. 1
- [4] S. Ahuja. Blendermcp - blender model context protocol integration. <https://github.com/ahujaaid/blender-mcp>, 2025. Accessed: 2025-04-25. 3, 4
- [5] E. W. Anderson, K. C. Potter, L. E. Matzen, J. F. Shepherd, G. A. Preston, and C. T. Silva. A user study of visualization effectiveness using eeg and cognitive load. In *Computer graphics forum*, vol. 30, pp. 791–800. Wiley Online Library, 2011. 1
- [6] Anthropic. Introducing computer use, a new claude 3.5 sonnet, and more, 10 2024. Accessed: 2025-04-24. 2, 4
- [7] Anthropic. Introducing the model context protocol, 11 2024. Accessed: 2025-04-16. 1, 2
- [8] A. Bendeck and J. Stasko. An empirical evaluation of the gpt-4 multimodal language model on visualization literacy tasks. *IEEE Transactions on Visualization and Computer Graphics*, 31(1):1105–1115, 2025. doi: 10.1109/TVCG.2024.3456155 2
- [9] M. Berger and S. Liu. The visualization judge: Can multimodal foundation models guide visualization design through visual perception? In *2024 IEEE Evaluation and Beyond-Methodological Approaches for Visualization (BELIV)*, pp. 60–70. IEEE, 2024. 2
- [10] T. Gao, M. Dontcheva, E. Adar, Z. Liu, and K. G. Karahalios. DataTone: Managing Ambiguity in Natural Language Interfaces for Data Visualization. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology*, pp. 489–500. ACM, Charlotte NC USA, Nov. 2015. doi: 10.1145/2807442.2807478 2
- [11] J. Han and C. Wang. Ssr-tyd: Spatial super-resolution for time-varying data analysis and visualization. *IEEE Transactions on Visualization and Computer Graphics*, 28(6):2445–2456, 2020. 2
- [12] K. Hu, M. A. Bakker, S. Li, T. Kraska, and C. Hidalgo. Vizml: A machine learning approach to visualization recommendation. In *Proceedings of the 2019 CHI conference on human factors in computing systems*, pp. 1–12, 2019. 2
- [13] S. Jeong, J. Li, C. R. Johnson, S. Liu, and M. Berger. Text-based transfer function design for semantic volume rendering. In *2024 IEEE Visualization and Visual Analytics*, pp. 196–200. IEEE, 2024. 3, 4
- [14] N. W. Kim, H.-K. Ko, G. Myers, and B. Bach. Chatgpt in data visualization education: A student perspective. In *2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, pp. 109–120. IEEE, 2024. 2
- [15] D. Koop, C. Scheidegger, S. Callahan, J. Freire, and C. Silva. VisComplete: Automating Suggestions for Visualization Pipelines. *IEEE Transactions on Visualization and Computer Graphics*, 14(6):1691–1698, Nov. 2008. doi: 10.1109/TVCG.2008.174 2
- [16] Z. Li, H. Miao, X. Yan, V. Pascucci, M. Berger, and S. Liu. See or recall: A sanity check for the role of vision in solving visualization question answer tasks with multimodal llms, 2025. 2
- [17] S. Liu, H. Miao, Z. Li, M. Olson, V. Pascucci, and P.-T. Bremer. AVA: Towards Autonomous Visualization Agents through Visual Perception-Driven Decision-Making. *Computer Graphics Forum*, 43(3):e15093, 2024. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.15093>. doi: 10.1111/cgf.15093 2
- [18] P. Ljung, J. Krüger, E. Groller, M. Hadwiger, C. D. Hansen, and A. Ynnerman. State of the art in transfer functions for direct volume rendering. *Computer Graphics Forum*, 35(3):669–691, 2016. doi: 10.1111/cgf.12934 1
- [19] Y. Lu, K. Jiang, J. A. Levine, and M. Berger. Compressive neural representations of volumetric scalar fields. In *Computer Graphics Forum*, vol. 40, pp. 135–146. Wiley Online Library, 2021. 2
- [20] T. Mallick, O. Yildiz, D. Lenz, and T. Peterka. Chatvis: Automating scientific visualization with a large language model. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 49–55. IEEE, 2024. 2
- [21] A. Narechania, A. Srinivasan, and J. Stasko. NL4DV: A Toolkit for generating Analytic Specifications for Data Visualization from Natural Language queries. *IEEE Transactions on Visualization and Computer Graphics (TVCG)*, 2020. doi: 10.1109/TVCG.2020.3030378 2
- [22] OpenAI. Introducing operator, January 2025. Accessed: 2025-04-24. 2
- [23] G. Ouyang, J. Chen, Z. Nie, Y. Gui, Y. Wan, H. Zhang, and D. Chen. nvagent: Automated data visualization from natural language via collaborative agent workflow. *arXiv preprint arXiv:2502.05036*, 2025. 2
- [24] A. Plaats, M. van Duijn, N. van Stein, M. Preuss, P. van der Putten, and K. J. Batenburg. Agentic large language models, a survey. *arXiv preprint arXiv:2503.23037*, 2025. 2
- [25] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pp. 8748–8763. PMLR, 2021. 3
- [26] V. Setlur, S. E. Battersby, M. Tory, R. Gossweiler, and A. X. Chang. Eviza: A natural language interface for visual analysis. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology*, UIST '16, p. 365–377. Association for Computing Machinery, New York, NY, USA, 2016. doi: 10.1145/2984511.2984588 2
- [27] P.-P. Vázquez. Are llms ready for visualization?, 2024. 2
- [28] H. W. Wang, M. Gordon, L. Battle, and J. Heer. Dracogpt: Extracting visualization design preferences from large language models. *IEEE Transactions on Visualization and Computer Graphics*, 2024. 2
- [29] L. Wang, S. Zhang, Y. Wang, E.-P. Lim, and Y. Wang. Llm4vis: Explainable visualization recommendation using chatgpt. *arXiv preprint arXiv:2310.07652*, 2023. 2
- [30] Y. Wang, Z. Hou, L. Shen, T. Wu, J. Wang, H. Huang, H. Zhang, and D. Zhang. Towards Natural Language-Based Visualization Authoring. *IEEE Transactions on Visualization and Computer Graphics*, pp. 1–11, 2022. doi: 10.1109/TVCG.2022.3209357 2
- [31] A. Wu, Y. Wang, X. Shu, D. Moritz, W. Cui, H. Zhang, D. Zhang, and H. Qu. Ai4vis: Survey on artificial intelligence approaches for data visualization. *IEEE Transactions on Visualization and Computer Graphics*, 28(12):5049–5070, 2021. 2
- [32] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023. 4
- [33] Y. Xie, Y. Luo, G. Li, and N. Tang. Haichart: Human and ai paired visualization system. *arXiv preprint arXiv:2406.11033*, 2024. 2
- [34] Z. Xu and E. Wall. Exploring the capability of llms in performing low-level visual analytic tasks on svg data visualizations. In *2024 IEEE Visualization and Visual Analytics (VIS)*, pp. 126–130, 2024. doi: 10.1109/VIS55277.2024.00033 2
- [35] Z. Yang, L. Li, K. Lin, J. Wang, C.-C. Lin, Z. Liu, and L. Wang. The dawn of llms: Preliminary explorations with gpt-4v (ision). *arXiv preprint arXiv:2309.17421*, 9, 2023. 2
- [36] Z. Yang, Z. Zhou, S. Wang, X. Cong, X. Han, Y. Yan, Z. Liu, Z. Tan, P. Liu, D. Yu, et al. Matplotagent: Method and evaluation for llm-based agentic scientific data visualization. *arXiv preprint arXiv:2402.11453*, 2024. 2
- [37] Y. Ye, J. Hao, Y. Hou, Z. Wang, S. Xiao, Y. Luo, and W. Zeng. Generative ai for visualization: State of the art and future directions. *Visual Informatics*, 2024. 2